

ROOT & SHOOT

GAME DESIGN CASE STUDY

A survivors-like built around one simple choice: keep moving, or stop and fight.

MOVE OR SHOOT. PICK ONE.

The player is a tree. To attack, they have to plant their roots into the ground, giving up movement for damage. Most of the project grew out of asking how enemies, upgrades, difficulty, and progression could make that one decision more interesting.



Top-down arena survival • Godot 4.7 • 3 levels / 7 rounds / 30 upgrades • Solo designer & developer

Design Goals

Before I started adding a bunch of systems, I wanted to be clear about what kind of game I was making. One of my biggest goals was keeping the difficulty on the more approachable side. I still wanted rooting at the wrong time to be dangerous, but I also wanted players to have enough breathing room to try upgrades, find synergies, and actually enjoy the build they put together.

- **Approachable difficulty** — Keep the game on the easier side so players have room to try upgrades, find synergies, and enjoy seeing their build come together.
- **One strong core mechanic** — Keep rooting as the simple idea that ties combat, enemies, upgrades, and positioning together.
- **Build experimentation** — Make upgrades lead to recognizable playstyles instead of only making the numbers bigger.
- **Readable combat** — Keep enemy tells consistent so players can read what is happening even when the arena gets busy.
- **Controlled complexity** — Start simple, then gradually introduce more interesting upgrades and enemy combinations.

- **Progression through variety** — Let unlocks add new ways to build the character instead of mostly giving permanent stat boosts.

DIFFICULTY SHOULD SUPPORT THE SYSTEMS, NOT HIDE THEM.

Bad root timing should still get you in trouble, but I wanted the game to be forgiving enough that players could focus on experimenting with builds and seeing how the systems work together.

01 – Designing Around Rooting

The main mechanic is simple: you can move or you can shoot, but you cannot do both at the same time. While moving, you can kite enemies and make space. When you root, you can finally attack, but now you are committing to that position until you uproot.

KITE -> CREATE SPACE -> ROOT -> FIRE -> UPROOT

I wanted the basic rhythm to be about creating little openings for yourself. You kite for a while, make some room, root for a quick burst of damage, and get moving again before the enemies close in.

- **Slight root slide** — Rooting does not stop the player instantly. The remaining momentum makes positioning something the player has to anticipate.
- **Manual aim** — The rooted window should test more than timing. The player still has to aim well once they commit to standing still.
- **Dash stays a movement tool** — Dash is blocked while rooted so it cannot become a free undo button for a bad commitment.
- **Upgrades should change the decision** — The upgrades I like most change how the player approaches rooting: root longer, root closer, root less often, or make each root more valuable.



02 – Upgrades Should Change How You Root

The upgrade system is a big part of the replayability, but I did not want it to just be a pile of stat boosts. I wanted upgrades to start pushing the player toward different ways of playing and, ideally, change how they think about rooting.

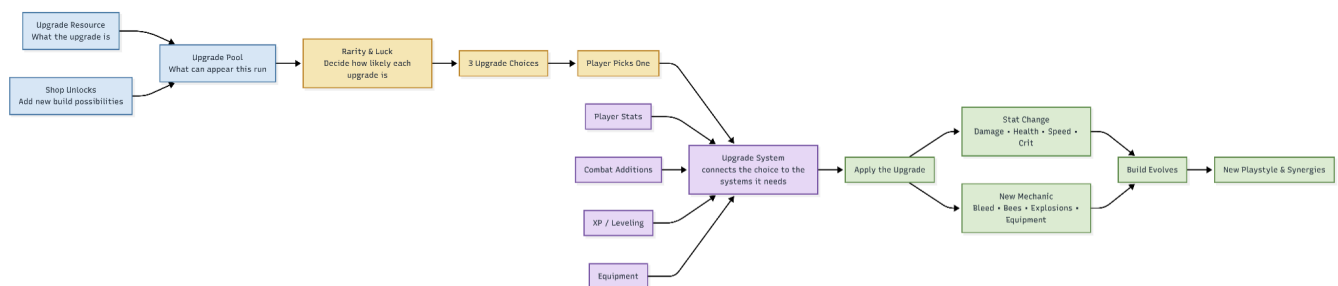
- **Crit / Burst** — Damage, Crit, Attack Speed, and Heavy Hitter reward short, deliberate damage windows.
- **Bleed / DoT** — Bleed and faster attacks let the player tag enemies, uproot, and keep getting value while kiting.
- **Thorns / Tank** — Health, Thorns, Pickup Radius, and regen let the player turn rooting near the crowd into an actual strategy.
- **Mobility** — Movement Speed, Ice Skates, Dash, and Teleporter reduce how much time the player has to stay committed.
- **Explosions / Bees** — Area effects make a single root control more of the arena instead of only increasing direct DPS.
- **Risk / Reward** — Plunge, Spicy Food, and Book offer stronger scaling while asking the player to accept more danger or delay immediate power.



The Upgrade Pipeline

I built the upgrades as reusable Resources in Godot. The technical side is useful, but the main design benefit is that it keeps the system flexible. I can add or remove upgrades, adjust rarity, and introduce new build ideas without having to redo the whole selection system.

Upgrade Pipeline — Game Design View



The technical structure supports a simple player-facing goal: turn a level-up choice into a meaningful change in playstyle.

DESIGN QUESTION Why use a modular Resource-based system?

Because I wanted the content to stay easy to tune. The design can grow by adding or removing upgrade possibilities while the same selection flow continues to work.

03 – Rarity as Run Pacing

I do not really look at rarity as just 'this upgrade is stronger.' I use it more as a way to pace the run. Early on, the player sees more straightforward choices. As the run goes on and a build starts taking shape, Rare and Legendary upgrades become more likely to show up.

```
t = clamp(upgrades_taken_this_level / 14, 0, 1)
```

```
rarity_weight = lerp(start_weight, end_weight, t)
```

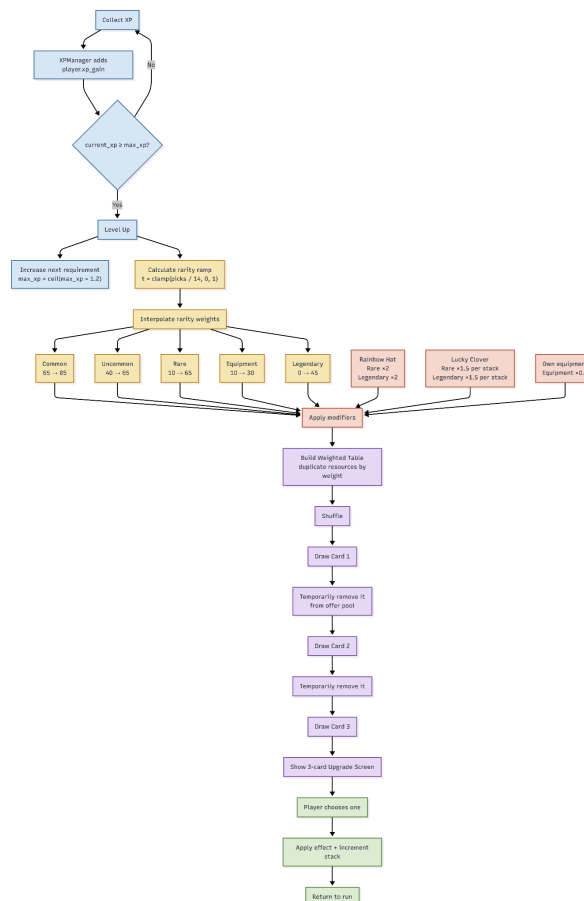
Legendary starts at weight 0 and rises to 45; Rare rises from 10 to 65.

This helps the run build up naturally. The player gets time to understand what is going on and start a build before the more dramatic upgrades enter the picture. It also makes those later choices feel a little more exciting.

A deliberate quirk

Common weight also rises from 65 to 85. The goal is not to replace Commons late in a level, but to let more exciting options join them while keeping reliable upgrades relevant.

XP, Rarity & Upgrade Selection



The full implementation is more detailed than the player needs to know, but it shows how the pacing rules feed the three-card draft.

04 – Making Stats Interact

I wanted the damage math to stay predictable even when a lot of upgrades start stacking together. Permanent bonuses build off base damage, while effects like Spicy Food can change during the run depending on what is happening.

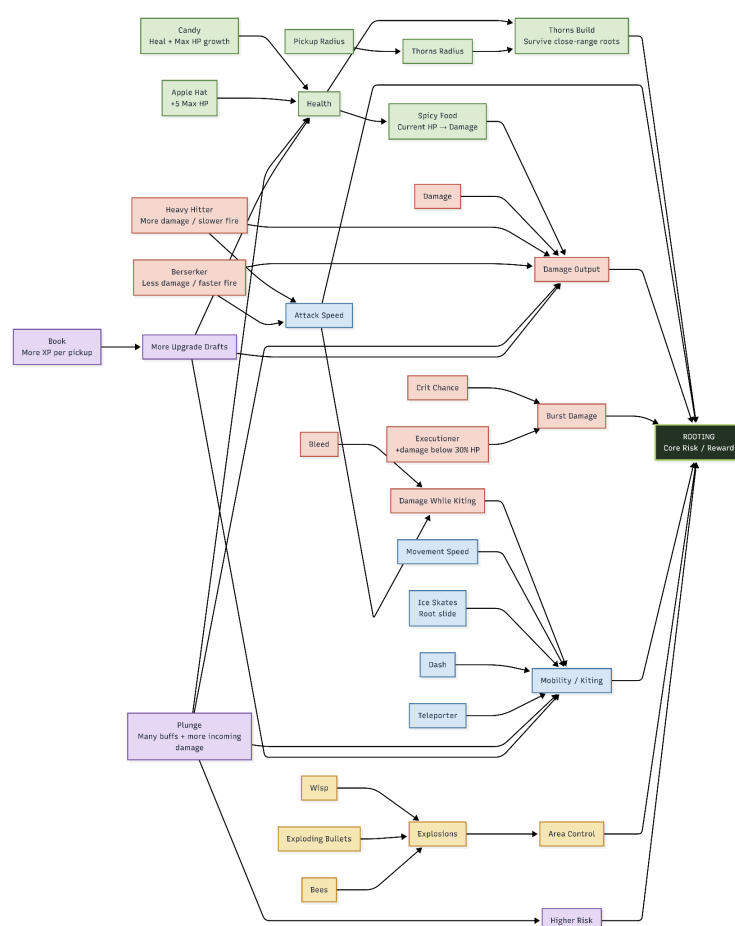
$$\text{Base Damage} = \text{Starting Damage} \times 1.2 \text{ (Damage Upgrade)} \times 1.3 \text{ (Heavy Hitter Upgrade)} \times 0.8 \text{ (Berzerker Upgrade)} \times 1.15 \text{ (Plunge Upgrade)}$$

$$\text{Live Damage} = \text{Base Damage} + \text{floor}(\text{Current HP} / 6)$$

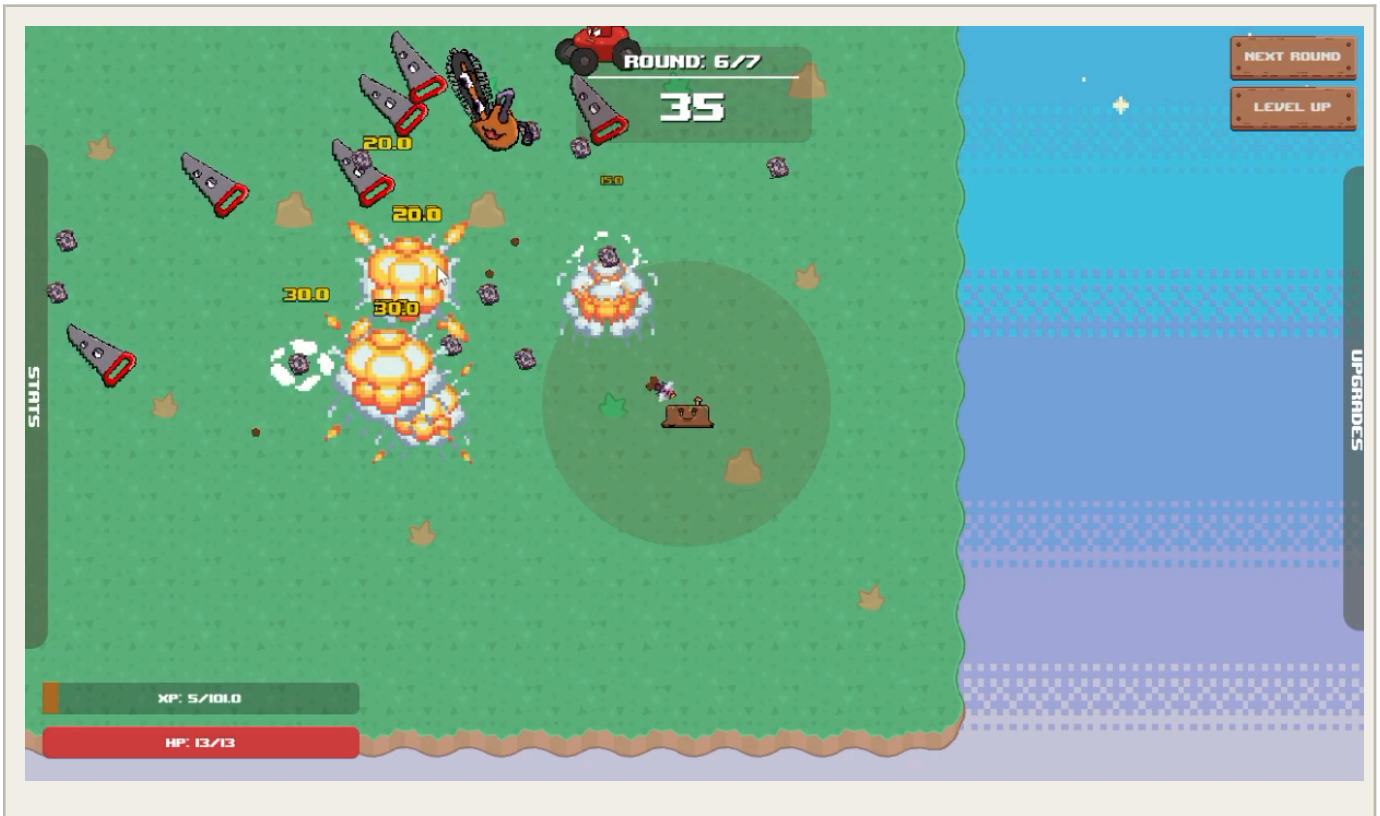
The health bonus only applies when Spicy Food is owned.

The formulas are useful, but the part I care about more is what they do to the player's choices. Heavy Hitter gives you more damage but slows your shots. Berserker pushes in the other direction. Spicy Food makes health part of your damage, so taking a hit also hurts your offense. Pickup Radius even increases the size of Thorns, which lets a normally simple utility stat become part of an actual build.

Build Synergy Web



Stats are useful when they connect to a playstyle rather than existing as isolated bonuses.



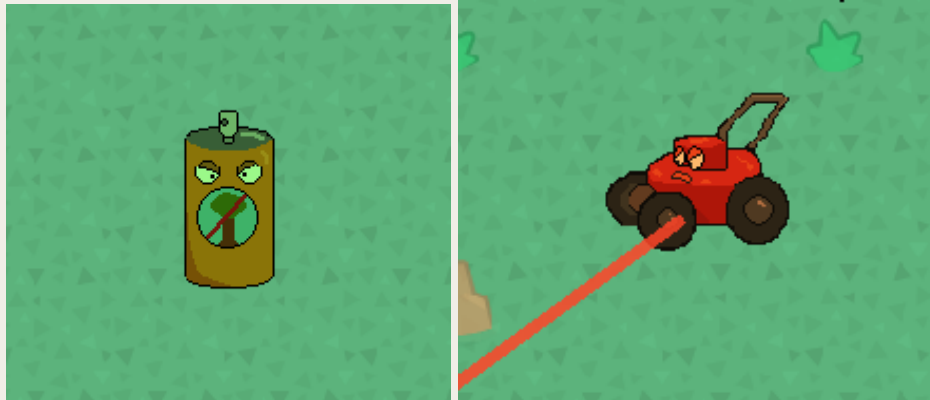
05 – Enemies Are Designed to Interrupt the Root Window

I designed the enemies around one main question: what can make the player think twice about rooting right now? Basic enemies keep pressure on you, while the special enemies each add a different reason to move, reposition, or cut a root short.

- **Bomber — punish camping** — It can catch a rooted player. A short root is survivable; staying planted too long is not.
- **Ranged — force movement** — Its stronger ranged attack gives the player another reason to leave a comfortable position.
- **Charger — test reading** — A growing aim line gives a clear warning before the dash, so the player has to read the threat before committing again.
- **Bosses — remix the lesson** — Area denial, spread fire, and high-speed charges turn the same root decision into larger encounters.

Shared telegraph language

Threats follow the same basic grammar: tell, wait, strike. Color communicates danger, and a line communicates direction.



DESIGN QUESTION Why does the Charger phase through other enemies during the dash?

Because I want the telegraph to be something the player can trust. If they read the aim line correctly, another enemy should not randomly change the charge and make the hit feel unfair.

06 – Difficulty Is Three Dials, Not One Slider

Since I wanted the game to stay on the easier side, I did not want later rounds to just crank enemy health through the roof. I still wanted the pressure to increase, but I wanted players to have room to enjoy the build they had put together. I ended up looking at difficulty through three main things: how many enemies there are, what kinds of enemies are showing up, and how durable they are.

- **Density** — Spawn timing controls how much space the player has to create a safe root window.
- **Variety** — Bombers, Ranged enemies, and Chargers add new problems instead of only adding more bodies.
- **Durability** — Enemy health rises each round, but the curve stays modest so a weaker build can still reach the end.

$$\text{Enemy HP} = \text{ceil}(\text{base_hp} \times (1 + 0.15 \times (\text{round} - 1)))$$

Round 1 = 1.0x; Round 7 = 1.9x.

Enemy count peaks around the middle of the level, then comes down a bit as stronger and more varied enemies start showing up. I liked that more than having every round simply be a bigger version of the last one.

07 – Meta Progression Sells Possibility, Not Power

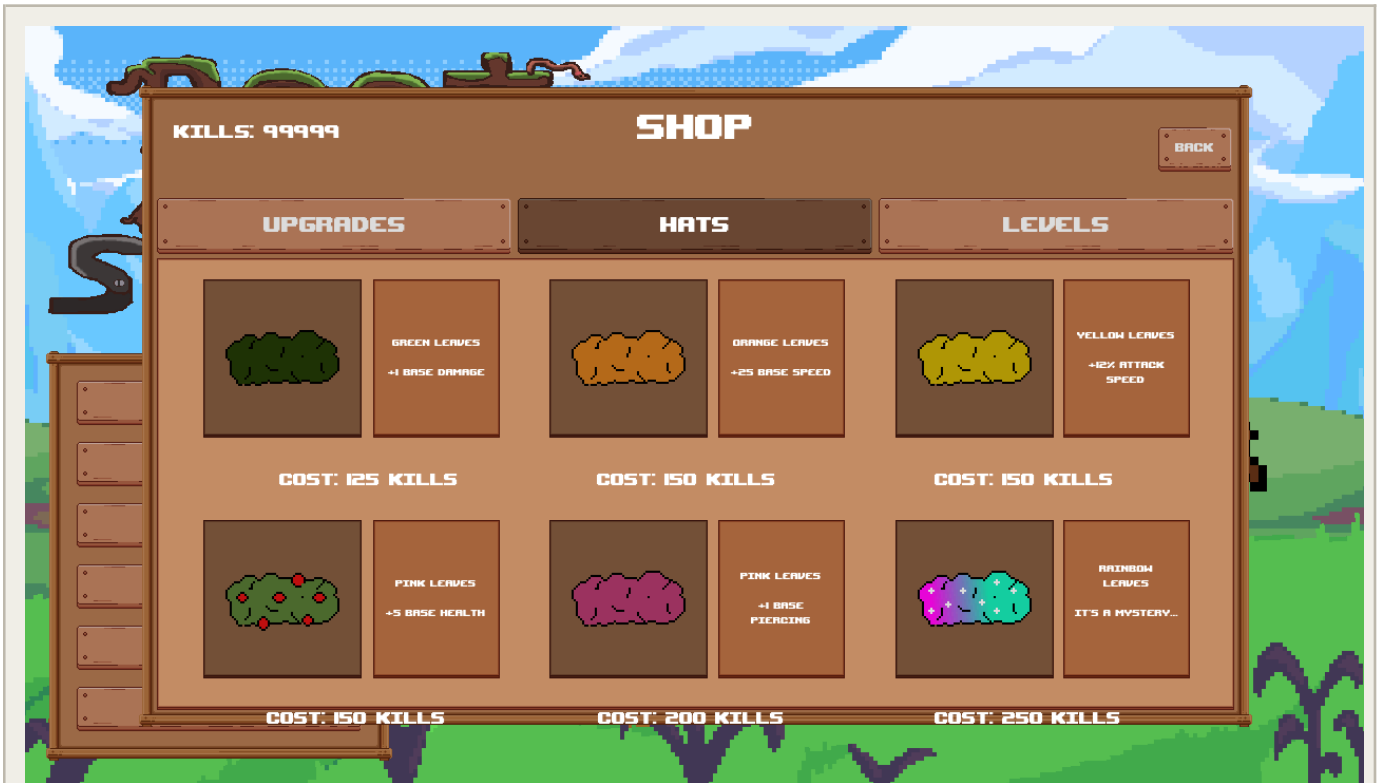
Kills give the player currency to spend between runs, but I did not want the shop to just hand out permanent damage and health upgrades. Most of the important unlocks add new upgrades to the pool instead.

23 default upgrades -> 29 fully unlocked

Crit, Bleed, Bees, Pierce, Sniper, and Exploding Bullets enter the pool only after being purchased.

So buying Crit does not simply make every future run stronger; it makes crit builds possible. I like that because the starting pool stays easier to understand, while the player gradually adds more possibilities as they keep playing.

Hats provide the smaller starting-choice axis. Green offers straightforward damage, Orange supports kiting, Apple supports health-based builds, and Rainbow changes the odds of seeing Rare and Legendary upgrades.



DESIGN QUESTION Why not sell permanent stat upgrades?

I wanted progression to give the player more options without making earlier runs feel pointless. Unlocking a new kind of build is more interesting to me than just starting every run with bigger numbers.

08 – Bosses Test What the Player Learned

I wanted the bosses to feel like bigger tests of things the player had already been learning. The Flame Boss controls space, the Ranged Boss keeps constant projectile pressure on you, and the Charger Boss makes finding a safe place to root much harder.

All three can appear in the final round, but I only allow one boss to be alive at a time. I wanted it to feel like a gauntlet, not a messy pile-up where the player cannot tell what is happening.



09 — What I Took Away From the Project

- **Protect the core tension** — If rooting is the game, convenience features should not quietly remove the risk that makes rooting interesting.
- **Make systems reinforce each other** — Health becoming damage, Pickup Radius growing Thorns, and enemy telegraphs interacting with root timing make the systems feel connected.
- **Use pacing to teach complexity** — Rarity ramps and shop unlocks let the decision space expand after the player understands the basics.
- **Tune pressure in different ways** — Changing density, enemy variety, and durability separately creates more interesting waves than one global difficulty multiplier.
- **Readability beats novelty** — A consistent threat language makes new enemies easier to understand when the screen gets busy.
- **Let progression create choices** — I prefer unlocks that make new strategies possible instead of permanent upgrades that simply make the player stronger.

DESIGN QUESTION What does Root & Shoot show about me as a designer?

For me, the project is really about taking one simple rule and seeing how much I can build around it. I wanted combat, upgrades, enemies, bosses, difficulty, and progression to all connect back to the same basic decision instead of feeling like separate systems.